FICHA 4:
Enunciado prueba parcial 4

Listas, diccionarios, funciones

Sigla	Asignatura	Experiencia de Aprendizaje
FPY1101	Fundamentos de Programación	EA3: Colecciones y funciones en Python
Tiempo	**Modalidad de Trabajo**	**Indicadores de logro**
2 h	Individual	IL 3.1 al IL 4.2



Resolver

Desarrolla un programa en Python que implemente un sistema de gestión de tareas pendientes, donde todo el comportamiento se organice mediante funciones bien definidas. El programa debe incluir un menú interactivo, validaciones de entrada, operaciones lógicas (decisiones y comparaciones) y uso de funciones separadas.

1. Datos que debe manejar el sistema

El sistema trabaja con una colección de tareas. Esta colección debe existir desde que el programa inicia y estar disponible durante toda la ejecución. Cada vez que se agrega una tarea, se incorpora a esa colección como un nuevo elemento.
Cada tarea se representa como un conjunto de campos asociados: descripción, prioridad, tiempo estimado y un indicador de si está completada o no. La siguiente tabla resume los campos de cada registro:

Campo	Qué representa	Restricciones de validación
`"descripcion"`	Descripción de la tarea	No vacía ni solo espacios en blanco
`"prioridad"`	Nivel de prioridad de la tarea	Número entero entre 1 y 10 (incluidos)
`"tiempo_estimado"`	Tiempo estimado para completar la tarea (en horas)	Número decimal mayor que cero
`"completada"`	¿La tarea ha sido completada?	**False al registrar.** No se valida ni se solicita al usuario: el sistema lo asigna

		automáticamente. Su valor puede cambiar a True cuando se ejecute la opción 4 (Actualizar estado), según la prioridad de la tarea.

Cada diccionario se guarda dentro de una lista. La lista es la colección general; los diccionarios son las tareas individuales dentro de ella. El programa comienza con la lista vacía y la va llenando a medida que se agregan registros.

2. Lo que debe hacer el sistema

El sistema se controla desde un menú que aparece en pantalla cada vez que el usuario termina una acción. El usuario elige una opción numérica, el programa ejecuta la tarea correspondiente y vuelve a mostrar el menú. Esto se repite hasta que el usuario elige salir.

El menú tiene seis opciones:

```
========== MENÚ PRINCIPAL ==========
1. Agregar tarea
2. Buscar tarea
3. Eliminar tarea
4. Actualizar estado
5. Mostrar tareas
6. Salir
====================================
```

Para implementar este comportamiento debes definir dos funciones separadas: una que muestre las opciones en pantalla (sin recibir nada ni retornar nada) y otra que lea y retorne la opción elegida por el usuario (sin recibir nada, retornando el número validado). Ambas funciones deben invocarse en cada vuelta del ciclo.

A continuación, se describe qué debe ocurrir al elegir cada opción:

Opción 1 - Agregar tarea:

El sistema solicita al usuario la descripción, la prioridad y el tiempo estimado de la tarea. Antes de guardar el registro, verifica que cada dato cumpla su condición:

- La descripción no puede estar vacía ni ser solo espacios en blanco.
- La prioridad debe ser un número entero entre 1 y 10.
- El tiempo estimado debe ser un número decimal mayor que cero.

Si algún dato no cumple la condición, el sistema informa al usuario y no registra la tarea. Solo cuando todos los datos son válidos se crea el diccionario y se agrega a la lista.

Para implementar esta opción debes definir una función que reciba la lista como parámetro. Dentro de ella se solicitan los datos al usuario y se llama a una función de validación distinta para cada campo. Los mensajes de error se muestran en esta función, no dentro de las validaciones.

Opción 2 - Buscar tarea:

El sistema solicita una descripción de la tarea al usuario y recorre la lista buscando un registro cuyo campo descripción coincida exactamente con el ingresado. Si lo encuentra, muestra la posición en la que está y sus datos. Si no existe ningún registro con esa descripción, informa al usuario.

Para implementar esta opción debes definir una función que reciba la lista y la descripción a buscar como parámetros. La función recorre la lista y retorna la posición del registro encontrado, o -1 si no existe. Es el programa principal quien recibe ese valor y decide qué hacer con él: si la posición es válida, muestra los datos de la tarea en esa posición; si es -1, muestra el mensaje de no encontrado.

Opción 3 - Eliminar tarea:

El sistema solicita la descripción de la tarea que se desea eliminar. Para localizarlo, llama a la función de búsqueda definida en la opción anterior, pasándole la lista y la descripción ingresada. Si la función retorna una posición válida, el sistema elimina el registro en esa posición. Si retorna -1, informa al usuario con el siguiente mensaje:

La tarea 'descripcion' no se encuentra registrada.

Opción 4 - Actualizar estado:

El sistema recorre la lista completa de tareas y actualiza el campo "completada" de cada registro según su prioridad: si la prioridad es mayor o igual a 5, el campo pasa a True; si es menor, queda en False. Esta operación afecta a todos los registros de la lista sin excepción.

Para implementar esta opción debes definir una función que reciba la lista como parámetro y aplique esa regla a cada elemento.

Opción 5 - Mostrar tareas:

El sistema primero actualiza el estado de todas las tareas haciendo el llamado a la función anterior, luego recorre la lista mostrando los datos de cada tarea. El formato de salida es el siguiente:

=== LISTA DE TAREAS ===

Descripción: Actualizar página web
Prioridad: 8
Tiempo estimado: 4.5
Estado: COMPLETADA
****
Descripción: Revisar correos
Prioridad: 3
Tiempo estimado: 1.0
Estado: PENDIENTE
****

Opción 6 - Salir:
El sistema termina la ejecución de forma limpia, sin errores. El ciclo del menú se detiene y el programa finaliza con un mensaje de despedida:

“Gracias por usar el sistema. Vuelva Pronto”